

Parametrized and Exact Algorithms

Notes by Arjun Maneesh Agarwal

based on course by G. Philip

The main book will be Parametrized Algorithms by Cygan et. al (chapters 1 to 7, 13 and 15). Other than that < copy from moodle >.

Table of Contents

1. Philip's Wisdom	1
2. Brief Intro and Motivation	1
3. Definitions	3
4. Exact Exponential Algorithms	4
5. Kernalization	4

1. Philip's Wisdom

"Why do you want to solve an algorithms problem, probably to make more money. Or kill more people. Or maybe save more people."

2. Brief Intro and Motivation

Parametrized Algorithms starts where the first algorithms course ends: NP Completeness.

Most algorithms we saw in the algorithms course have complexity $O(n^c)$ for some constant c . This raises the question if that is true of all problems, sadly, this is where NP Hardness and Completeness comes in. While we don't know if we can solve them polytime, these NP Complete problems span thousands of fields and experts in their field have tried finding fast algorithms. As these problems are complete, all reduce to each other which implies that if one of them succeed, all of them succeed. Unfortunately, they have all failed, which is the sort of silence that speaks quite loudly.

So how do we cope with this? One is Approx Algorithms. The other idea is Random Algorithms. Another is heuristics and meta-heuristics (taken from physics and biology and what not).

In theory, NP Completeness implies all the problems are equally hard modulo some polynomial time translations. However, some problems were easier to solve in practice.

Example

Type Inference for Hindley-Milner type system is unconditionally not solvable in polynomial time (in the size n of the input programme) as it hard for the class

EXP (that is problems that can be solved in $O(2^{\text{poly}})$ time) (from Maivson, POPL 1990) and $P \subsetneq \text{EXP}$ (Time Hierarchy theorem)

However, we don't think about the size of programme while writing code. And the compiler happily (or sadly) infers the type quite quickly.

Theorem 2.1. *Type inference for Hindley-Milner Type system can be solved in $O(2^k n)$ time where*

- *n is the size of input programme,*
- *k is the nesting depth of types in the programmes*

(Lichtenstein and Porveli, POPL 1985)

The idea is to pick something that tends to something that is usually small in practice and limit the exponential blow up to that parameter.

So our roadmap is: pick a parameter k that could be natural or something more convoluted, try to find an algorithm of time complexity $O(f(k)n^c)$. If we succeed the problem is Fixed Parameter Tractable aka FPT.

Definition: Vertex Cover

Vertex cover of a graph G is a subset $S \subseteq V(G)$ such that $G \setminus S$ has no edges.

Definition: Vertex Cover (Standard Parametrization)

Input: Graph G , $k \in \mathbb{N}$

Parameter: k

Question: Does G have a vertex cover of size $\leq k$?

Remark

This can be thought of as choosing a subset as guard posts such that all the roads are guarded.

Also, please make sure to not confuse this with **Dominating Set** (which is choosing vertices to put 1 edge radius umbrellas to cover everyone).

While vertex cover is NP Complete, we can see a simple algorithm that runs in $\mathcal{O}^*(2^k)$ time (we can also do it in $\mathcal{O}^*(1.4656^k)$ time as we'll later see and in $\mathcal{O}^*(1.2529^k)$ as we shall not see).

The idea is that given an edge, one of the two vertices neighboring it have to be in the vertex cover. This is a local property and can be abused to get the algorithm.

 VERTEX COVER IN $O(2^k n)$ TIME

```

1 func existVertexCover( $G, k$ ):
2   |   pick edge  $uv \in V(G)$ 
3   |   return existVertexCover( $G \setminus \{u\}, k - 1$ ) OR existVertexCover( $G \setminus \{v\}, k - 1$ )

```

3. Definitions

Definition: Classical Language

Let Σ be a finite alphabet, for example $\Sigma = \{0, 1\}$ or $\Sigma = \text{ASCII}$. A classical language over Σ is $L \subseteq \Sigma^*$, for example $L = \{\text{All valid Haskell programmes}\}$.

Definition: Parametrized Language

A parametrized language over Σ is $L \subseteq \Sigma^* \times \mathbb{N}$, for example $L = \{(x, k) \mid x \text{ is a valid Haskell programme with } k \text{ parameters}\}$.
 For a fixed k , $L_k = \{(x, k) \mid (x, k) \in L\}$ is the k -th slice of L .

Definition: Slice-wise Polynomial Time (XP)

A parametrized language L is slice-wise polynomial time solvable (XP) if there exists an algorithm $\mathcal{A}$ and a computable function f such that:

- For all x, k , $\mathcal{A}$ runs in $\leq |x|^{f(k)}$ time on input (x, k)
- $(x, k) \in L \iff \mathcal{A}((x, k)) = \text{YES}$

Definition: Fixed Parameter Tractable (FPT)

A parametrized language L is fixed parameter tractable (FPT) if there exists an algorithm $\mathcal{A}$ and a computable function f such that:

- For all x, k , $\mathcal{A}$ runs in $\leq f(k) |x|^c$ time on input (x, k) where c is a constant independent of $|x|$ and k .
- $(x, k) \in L \iff \mathcal{A}((n, k)) = \text{YES}$.

For notational convenience, $f(k) |x|^c$ is abbreviated $\mathcal{O}^*(f(k))$.

One motivation could be this table:

$\frac{n^{k+1}}{2^k n}$	$n = 50$	$n = 100$	$n = 150$
$k = 2$	625	2500	5625
$k = 3$	15625	125000	421875
$k = 10$	10^{12}	8×10^{13}	3.7×10^{16}
$k = 20$	1.8×10^{26}	5×10^{30}	2.1×10^{35}

4. Exact Exponential Algorithms

An example could be Hamiltonian Path.

Definition: Hamiltonian Path

Input: Given a graph G and vertex $v \in V$

Question: Does there exist a path P starting at v which covers all vertices of G without repeating vertices or edges.

This problem is NP Hard, however using a DP on sets, we can do this in $\mathcal{O}^*(2^n)$ instead of the brute force $\mathcal{O}^*(n!)$.

Another example is Chromatic Number

Definition: Chromatic Number

Input: Given a graph G and $k \in \mathbb{N}$

Question: Can we properly color the graph G using k colors

While there is a $\mathcal{O}^*(2^k)$ algorithm, we can see an interesting and simple $\mathcal{O}^*(3^k)$ algorithm.

Consider all the subset of vertices and store all the independent sets (that means can be 1 colored). Now consider subsets of vertices and store all the ones that can be partitioned into independent sets (which will take 3^n time as we are basically 3 partitioning the vertices into $S = X \sqcup Y$ and $V \setminus S$) giving us the 2-colorable subsets. Repeat with 3 coloring and so on. This will give an $\underbrace{\mathcal{O}^*(2^n) + \mathcal{O}^*(3^n) + \mathcal{O}^*(3^n) + \dots + \mathcal{O}^*(3^n)}_{k \text{ terms}} = \mathcal{O}^*(3^n)$.

5. Kernalization

TODO. *Introduction with sorting and cycle detection idea*

Example : European Rail Network Problem

Given an input of 1, 40, 000 trains, 25, 000 stations and 16, 00, 000 single train stops (valid trains and station pairs).

Our question is if there is any subset T of trains, they wanted to find a smallest set S of stations such that each train T stops at atleast one of the stations in S .

This is roughly equivalent to the red-blue dominating set which is NP Hard. However, Karsten Werhe gave a set of reduction rules (similar to the cycle detection) that would split this into small cases which he could solve by hand.

TODO. *Section 4.6 of Fundamentals of Parametrized Complexity by Downey and Fellows*

This is a weird place as if we had a polynomial algorithm that could reduce the size of an NP problem by even 1 bit, we could just use the same algorithm to keep reducing it to 1 single bit (which would be the answer).

So what do we do?

TODO. *Parametrization of Kernel*

TODO. *Kernel of $\mathcal{O}^*(k^2)$ for vertex cover*

Theorem 5.1. *A problem admits an FPT algorithm if and only if it admits a kernelization.*